6.3 디플로이먼트

Deployment

- 배포할 때 사용하는 가장 기본적인 컨트롤러
- 레클리카세트를 관리하면서 앱 배포를 더 세밀하게 관리
- 파드 개수를 유지
- 롤링 업데이트와 롤백을 지원

1. 디플로이먼트 사용하기

- 디플로이먼트 설정 예(deployment/deployment-nginx.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx-deployment
  # annotations:
    # kubernetes.io/change-cause: version 1.10.1
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  template:
    metadata:
      labels:
        app: nginx-deployment
    spec:
      containers:
      - name: nginx-deployment
        image: nginx:1.14
        ports:
        - containerPort: 80
```

- .spec.replicas: 파드를 몇 개 실행할 것인지
- .spec.seletor.machlabels의 하위 필드는 .metadata.labels의 하위 필드와 같은 설정을 해야 한다.
- .spec.template.spec.containers[]: 실제 사용하려는 컨테이너의 이름과 이미지 정보가 있다.

```
$ kubectl get deploy,rs,pods
NAME                                READY   UP-TO-DATE   AVAIL
deployment.apps/nginx-deployment    3/3     3            3

NAME                                           DESIRED   CURRE
replicaset.apps/nginx-deployment-7ff96dcbcb    3         3

NAME                                       READY   STATUS    RES
pod/nginx-deployment-7ff96dcbcb-9287d      1/1     Running   0
pod/nginx-deployment-7ff96dcbcb-btb4z      1/1     Running   0
pod/nginx-deployment-7ff96dcbcb-rmk9l      1/1     Running   0
```

- 실행 결과
  - nginx-deployment: 디플로이먼트
  - nginx-deployment-7ff96dcbcb: 레플리카세트(숫자=레플리카세트를 구분하는 UUID 해시문자)
  - nginx-deployment-7ff96dcbcb-9287d: 레플리카세트가 관리하는 파드들

디플로이먼트가 관리하는 컨테이너 이미지 설정 정보의 업데이트

- **kubectl set 명령**
  - 직접 컨테이너 이미지를 지정
  - kubectl set deployment 디플로이먼트 컨테이너명=이미지:버전 (--record)
  - --record 버전관련숫자 확인
- **kubectl edit 명령**
  - 현재 디플로이먼트의 설정 정보를 연 다음 컨테이너 이미지 정보를 수정
- **kubeclt apply 명령**
  - 처음 적용했던 템플릿의 컨테이너 이미지 정보를 수정한 다음 kubectl apply 명령을 실행해서 변경

```
$ kubectl set image deploy nginx-deployment nginx-deployment=
deployment.apps/nginx-deployment image updated
$ kubectl get deploy,rs,po
NAME                                READY   UP-TO-DATE   AVAIL
deployment.apps/nginx-deployment    3/3     3            3

NAME                                           DESIRED   CURRE
replicaset.apps/nginx-deployment-56958679b6    0         0
replicaset.apps/nginx-deployment-8947cc5b7     3         3

NAME                                     READY   STATUS    REST
pod/nginx-deployment-8947cc5b7-dwwtj     1/1     Running   0
pod/nginx-deployment-8947cc5b7-jh2mm     1/1     Running   0
pod/nginx-deployment-8947cc5b7-ns78x     1/1     Running   0
```

```
$ kubectl edit deploy nginx-deployment
deployment.apps/nginx-deployment edited
$ kubectl get po
NAME                                READY   STATUS    RESTARTS
nginx-deployment-7b7d8db48-cvqr2    1/1     Running   0
nginx-deployment-7b7d8db48-hcz7r    1/1     Running   0
nginx-deployment-7b7d8db48-tpkck    1/1     Running   0
$ kubectl describe pods nginx-deployment-7b7d8db48-cvqr2
```

```
$ vi deployment-nginx.yaml
$ kubectl apply -f deployment-nginx.yaml
deployment.apps/nginx-deployment configured
$ kubectl get pods
```

---

2. 디플로이먼트 롤백하기

kubectl rollout history deploy 디플로이먼트명

- 컨테이너 이미지 변경한 내역 확인

```
$ kubectl rollout history deploy nginx-deployment
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
```

- 리비전이 여러 개 보인다. 현재 리비전은 3이다. 특정 리비전의 상세 내용을 확인하려면 **--revision=리비전숫자** 옵션을 사용

```
$ kubectl rollout history deploy nginx-deployment --revision=
^[[Ddeployment.apps/nginx-deployment with revision #3
Pod Template:
  Labels:   app=nginx-deployment
    pod-template-hash=7b7d8db48
  Containers:
   nginx-deployment:
    Image:  nginx:1.16
    Port:   80/TCP
    Host Port:  0/TCP
    Environment:    <none>
    Mounts: <none>
  Volumes:  <none>
```

kubectl rollout undo deploy 디플로이먼트명

- 바로 직전의 revision으로 돌려놓는다.

```
$ kubectl rollout undo deploy nginx-deployment
deployment.apps/nginx-deployment rolled back
```

```
$ kubectl get deploy,rs,pods
NAME                                READY   UP-TO-DATE   AVAIL
deployment.apps/nginx-deployment    3/3     3            3

NAME                                          DESIRED   CURRE
replicaset.apps/nginx-deployment-56958679b6   0         0
```

```
replicaset.apps/nginx-deployment-698c7647df   0         0
replicaset.apps/nginx-deployment-7b7d8db48    3         3

NAME                                      READY   STATUS    REST
pod/nginx-deployment-7b7d8db48-469bz      1/1     Running   0
pod/nginx-deployment-7b7d8db48-66rh9      1/1     Running   0
pod/nginx-deployment-7b7d8db48-9mfqk      1/1     Running   0
```

- kubectl rollout history deploy 디플로이먼트명
  - 리비전 숫자 확인
  - 리비전 2가 리비전 4로 변경

```
$ kubectl rollout history deploy nginx-deployment
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
1         <none>
3         <none>
4         <none>sudo usermod -aG docker $USER && newgrp docke
```

- 특정 리비전으로 실행 중인 파드를 되돌리려면 --to-revision=리비전숫자 옵션을 사용

```
$ kubectl rollout undo deploy nginx-deployment --to-revision=
deployment.apps/nginx-deployment rolled back
```

- 참고로 되돌릴 수 있는 리비전 숫자는 디플로이먼트 템플릿의 .spec.revisionHistoryLimit 필드 값을 설정하면 된다. 기본값은 10이다.
- 앞에서 kubectl rollout history 명령을 실행했을 때 출력 결과 중 CHANGE-CAUSE 항목은 <none>이다. 이 항목은 해당 리비전의 주요 내용을 나타내는 부분이다.
- CHANGE-CAUSE 항목에 내용을 출력하려면 야물 파일에 .metadata.annotation 필드를 추가하면 된다.

```
metadata:
  name: nginx-deployment
  labels:
    app: nginx-deployment
```

```
  annotations:
    kubernetes.io/change-cause: version 1.10.1
```

- 변경 사항을 저장한 후 kubectl apply -f deployment-nginx.yaml 명령을 실행하면 다시 내역을 볼 수 있다.

--record 옵션

- 업데이트 시 직접 CHAGNE-CAUSE 옵션에 내역을 입력

```
kubectl set image deploy nginx-deployment nginx-deployment=ng
```

---

3. 파드 개수 조정하기

kubectl scale deploy 디플로이먼트명 - -replicas=파드개수

```
$ kubectl scale deploy nginx-deployment --replicas=5
deployment.apps/nginx-deployment scaled
cka@ubt:~/test$ kubectl get deploy,rs,pods
NAME                                READY   UP-TO-DATE   AVAIL
deployment.apps/nginx-deployment    5/5     5            5

NAME                                           DESIRED   CURRE
replicaset.apps/nginx-deployment-7cfdb99744    5         5

NAME                                       READY   STATUS    RES
pod/nginx-deployment-7cfdb99744-b4mlm      1/1     Running   0
pod/nginx-deployment-7cfdb99744-hr9c4      1/1     Running   0
pod/nginx-deployment-7cfdb99744-jc8dv      1/1     Running   0
pod/nginx-deployment-7cfdb99744-l6vcc      1/1     Running   0
pod/nginx-deployment-7cfdb99744-nmn7k      1/1     Running   0
```

---

4. 디플로이먼트 배포 정지, 배포 재개, 재시작하기

kubectl rollout pause deploy 디플로이먼트명

- 디플로이먼트 배포 정지

kubectl rollout resume deploy 디플로이먼트명

- 디플로이먼트 배포 재개

kubectl rollout restart deploy 디플로이먼트명

- 디플로이먼트 배포 재시작

---

5. 디플로이먼트 상태

배포 중에는 디플로이먼트 상탵가 변한다. 우선 진행(Processing)이었다가 성공이면 완료(Complete), 실패면 실패(failed) 상태로 바뀐다. **kubectl rollout status** 명령으로 배포 진행 상태를 확인할 수 있다.

- 상태가 진행으로 표시
  - 디플로이먼트가 새로운 레플리카세트를 만들 때
  - 디플로이먼트가 새로운 레플리카세트의 파드 개수를 늘릴 때
  - 디플로이먼트가 예전 레플리카세트의 파드 개수를 줄일 때
  - 새로운 파드가 준비 상태가 되거나 이용 가능한 상태가 되었을 때
- 완료로 표시(완료되면 종료 코드가 0으로 표시)
  - 디플로이먼트가 관리하는 모든 레플리카세트가 업데이트 완료되었을 때
  - 모든 레플리카세트가 사용 가능해졌을 때
  - 예전 레플리카세트가 모두 종료되었을 때
- 실패의 이유는 보통 다음과 같다.
  - 쿼터 부족
  - readinessProbe 진단 실패
  - 컨테이너 이미지 가져오기 에러
  - 권한 부족
  - 제한 범위 초과
  - 앱 실행 조건을 잘못 지정